Instituto Tecnológico de Costa Rica

Lenguajes de Programación

Proyecto #3

Presenta:

Esteban Rodríguez Vargas

Jeanpooll Domenech Cerdas

Profesor:

Allan Rodríguez Davila

Grupo:

60

Segundo Semestre 2026

Noviembre, 2026

Contenido

1. Descripción del problema
2. Manual de usuario
3. Pruebas de funcionalidad
4. Diseño del programa
5. Análisis de resultados
6. Análisis de predicados
7. Bitácora

1. Descripción del problema

Una academia de investigación criminal necesita una herramienta experimental que apoye a sus investigadores en el análisis de casos. Cada caso reúne información sobre personas, lugares, objetos, horarios, accesos, movimientos, evidencias y testimonios. El investigador no recibe la respuesta directamente: debe obtenerla mediante razonamiento lógico a partir de los datos disponibles.

El proyecto implementa este sistema mientras usamos el paradigma lógico. El caso se representa como una base de conocimiento de hechos en Prolog, y las conclusiones (quién tuvo acceso, quién tiene coartada, quién se contradice, quién es un posible sospechoso y por qué) se infieren con reglas. A diferencia de un programa tradicional, no se escribe el procedimiento para encontrar al responsable: Prolog lo determina mediante unificación y backtracking.

Además de la lógica en Prolog, el sistema ofrece una interfaz web hecha con React. La interfaz solo recibe parámetros del usuario, los traslada a Prolog a través de un servidor intermedio y muestra la respuesta, no contiene ninguna regla del caso.

1.1 Objetivos del proyecto

- Practicar habilidades de desarrollo de software.
- Ejercitar la toma de decisiones sobre el dominio del problema y de la solución.
- Aplicar los conceptos del paradigma lógico.
- Utilizar unificación y backtracking para encontrar múltiples soluciones.

1.2 Alcance

Predicados desarrollados:

- tiene_acceso/2, tiene_contradiccion/1, tiene_coartada/1, posible_sospechoso/1, posibles_responsables/1, explicaciones/2 y resolver_caso/0.
- estuvo/3 corresponde directamente a los hechos del mismo nombre de la base de conocimiento.

Predicados excluidos por indicación del profesor: estuvo_en_intervalo/4, evidencias_de/2, descartar/1 e investigar/1.

2. Manual de usuario

Esta sección explica cómo preparar, ejecutar y usar el sistema.

2.1 Requisitos

Programa	Uso
SWI-Prolog	Ejecuta la lógica del caso. Debe poder invocarse con el comando swipl.
Node.js (versión LTS)	Ejecuta el servidor intermedio y la interfaz web.
Navegador web	Muestra la interfaz.

2.2 Obtener el proyecto

1. En el repositorio de GitHub, pulsar **Code - Download ZIP**.
2. Descomprimir el archivo en una carpeta.
3. Comprobar que existan las carpetas programa/backend y programa/frontend.

2.3 Instalación de dependencias

Se hace una sola vez. En una terminal:

```
cd programa/backend
npm install
```

```
cd programa/frontend
npm install
```

Si el comando swipl no es reconocido, indicar la ruta de SWI-Prolog antes de arrancar el servidor. Por ejemplo, en PowerShell: $env:SWIPL="C:\Program Files\swipl\bin\swipl.exe".

2.4 Ejecución

Se necesitan **dos terminales** abiertas al mismo tiempo.

Terminal 1 – servidor (backend):

```
cd programa/backend
node server.js
```

Debe aparecer el mensaje Backend escuchando en http://localhost:3001.

Terminal 2 – interfaz (frontend):

```
cd programa/frontend
npm run dev
```

Abrir en el navegador la dirección que muestra la terminal, normalmente http://localhost:5173.

2.5 Uso de la interfaz

Elemento	Qué hace
Lista **Persona**	Selecciona a quién se aplican las consultas de la sección siguiente. Se llena con las personas del caso cargado.
Lugares con acceso	Muestra los lugares a los que la persona puede ingresar (tiene_acceso/2), sin repetidos.
¿Tiene coartada?	Responde Si o No (tiene_coartada/1).
¿Tiene contradicción?	Responde Si o No (tiene_contradiccion/1).
¿Es posible sospechoso?	Responde Si o No (posible_sospechoso/1).
Explicaciones	Lista, una por línea, las razones lógicas que aplican a la persona (explicaciones/2).
Posibles responsables	Muestra la lista de personas sospechosas del caso (posibles_responsables/1).
Resolver caso	Ejecuta resolver_caso/0 y muestra, en el recuadro amarillo, el reporte completo de cada responsable.

Si ocurre un error (servidor apagado, persona inexistente, Prolog sin responder), la interfaz muestra un mensaje en rojo que explica qué pasó.

2.6 Cambiar el caso

Los datos del caso están únicamente en programa/backend/prolog/hechos.pl. Para usar otro caso basta reemplazar el contenido de ese archivo (por ejemplo, con el de hechos_caso2.pl).

No hay que modificar predicados.pl. Después de reemplazarlo, se refresca la página de la interfaz para cargar la nueva lista de personas.

2.7 Uso directo en la consola de Prolog

También se puede consultar la lógica sin interfaz:

```
cd programa/backend/prolog
swipl predicados.pl

?- posibles_responsables(L).
?- explicaciones(bruno, E).
?- resolver_caso.
```

Al escribir una consulta que tiene varias soluciones, se pulsa, para ver la siguiente. Para salir: halt.

3. Pruebas de funcionalidad

Las pruebas siguen la guía Guia_de_Pruebas.md incluida en la carpeta de documentación. Se probó la lógica en la consola de Prolog con dos casos distintos, el servidor, la interfaz y los casos de error. Cada captura muestra la consulta o la acción y el resultado obtenido.

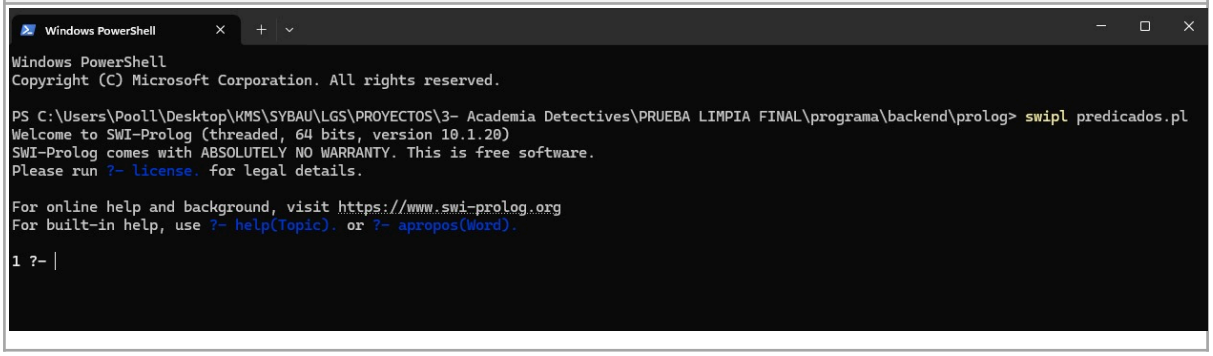
Los resultados detallados de la lógica en Prolog están también en el archivo pruebas_prolog.txt.

3.1 Lógica en Prolog – Caso 1 (museo)

Caso base: una desaparición en la bodega del museo a las 2000, con cinco personas.

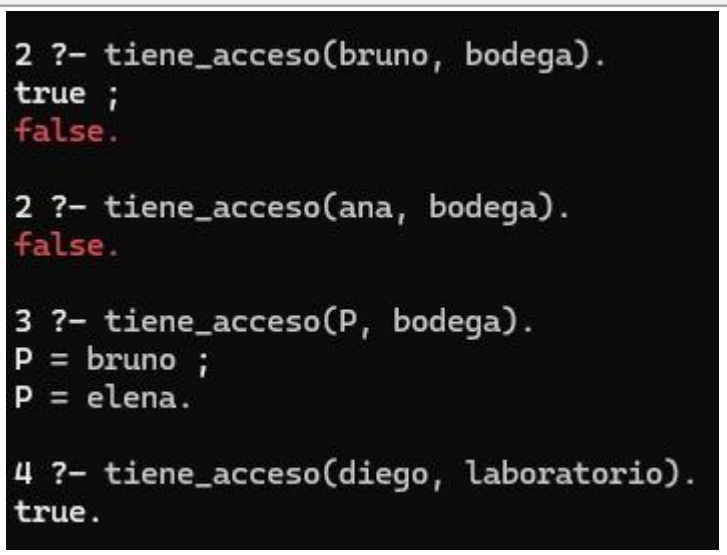
Captura C01 – Programa cargado sin errores

Resultado esperado: Prompt de SWI-Prolog sin mensajes de error ni advertencias.



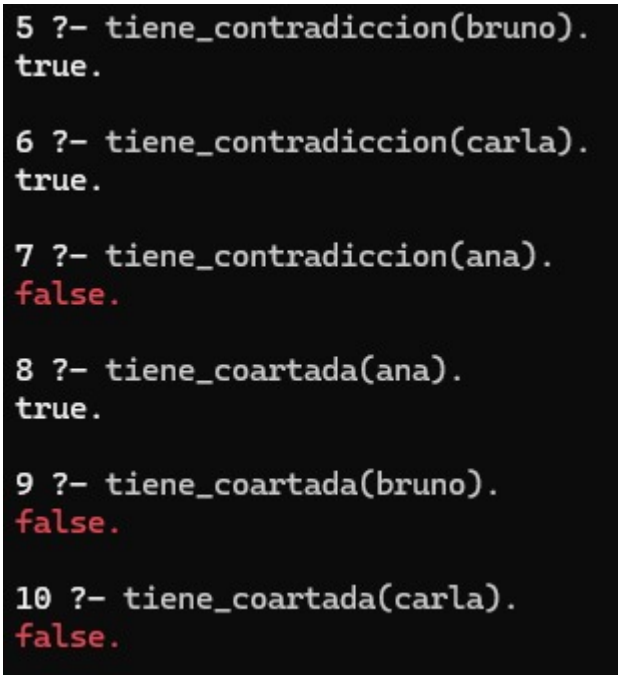
Captura C02 – Predicado tiene_acceso/2

Resultado esperado: true para bruno-bodega, false para ana-bodega, bruno y elena para la bodega, true para diego-laboratorio (por objeto).



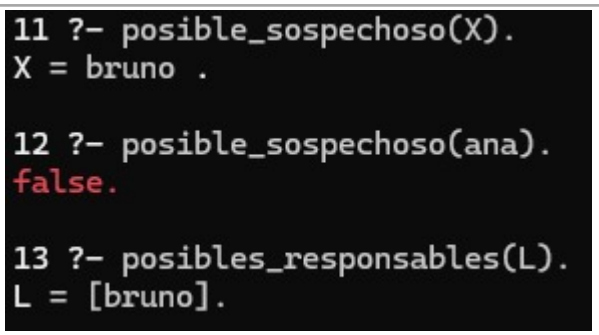
Captura C03 – Predicados tiene_contradiccion/1 y tiene_coartada/1

Resultado esperado: Contradicción en bruno y carla, no en ana, coartada en ana, no en bruno ni carla.



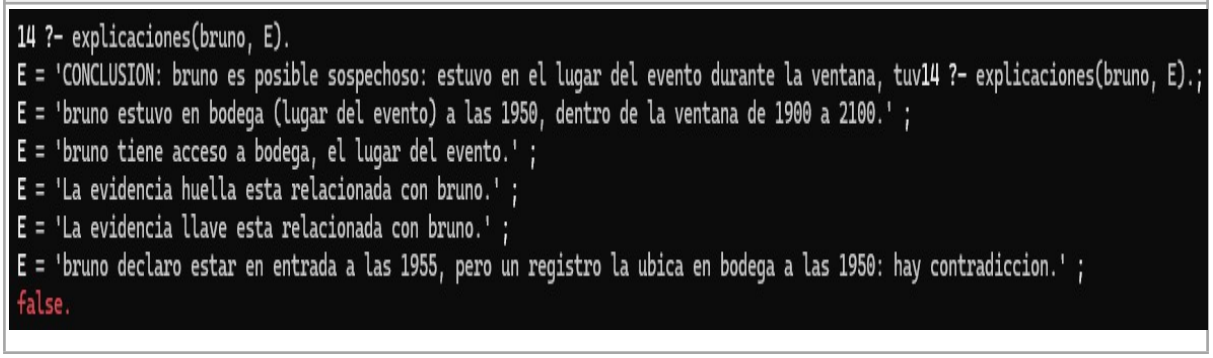
Captura C04 – Predicados posible_sospechoso/1 y posibles_responsables/1

Resultado esperado: X = bruno, L = [bruno].



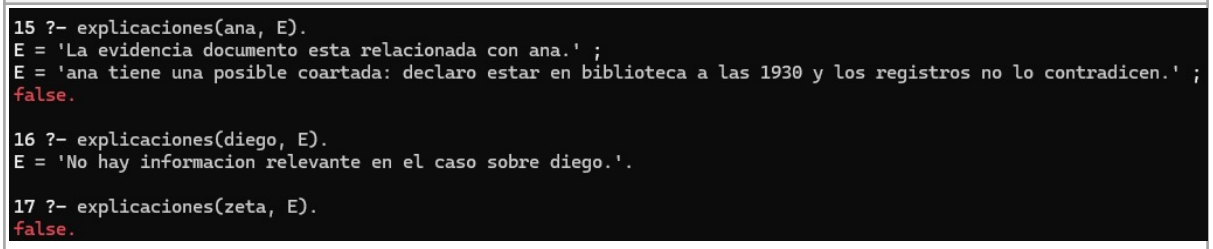
Captura C05 – Predicado explicaciones/2 para bruno

Resultado esperado: Seis explicaciones obtenidas por backtracking.



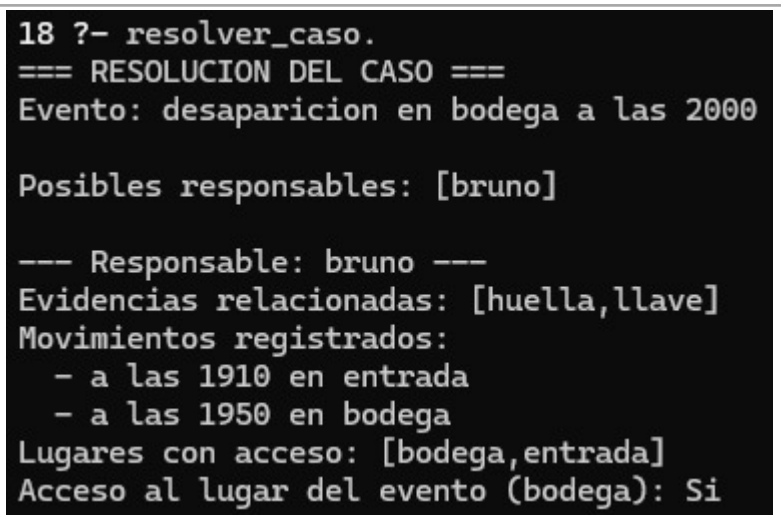
Captura C06 – explicaciones/2 para diego y para una persona inexistente

Resultado esperado: "No hay informacion relevante..." para diego, false para zeta.



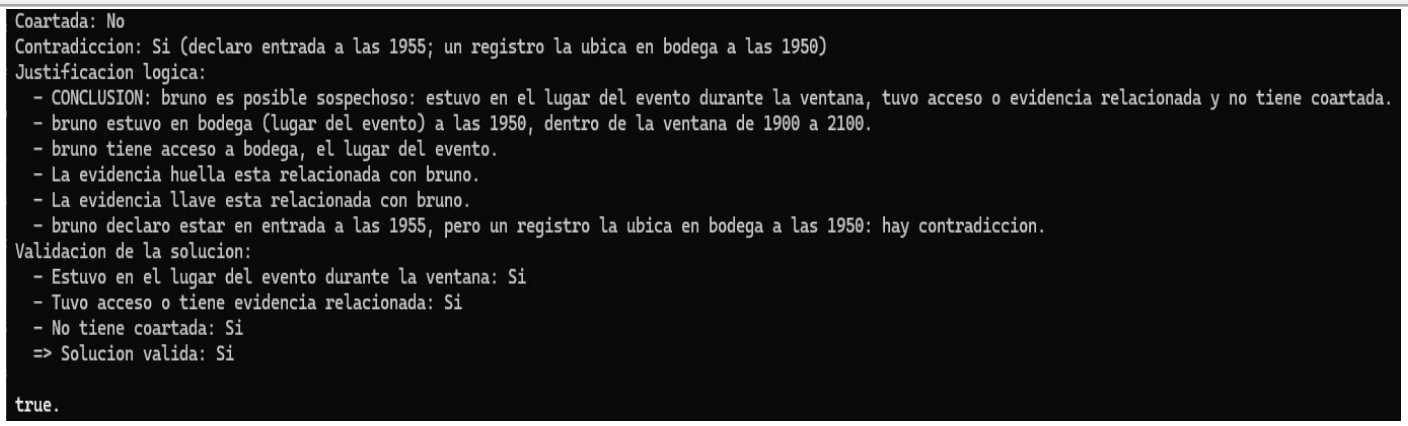
Captura C07 – resolver_caso/0 (primera parte)

Resultado esperado: Evento, responsables, evidencias, movimientos y accesos de bruno.



Captura C08 – resolver_caso/0 (segunda parte)

Resultado esperado: Coartada, contradicción, justificación y validación con "Solucion valida: Si".

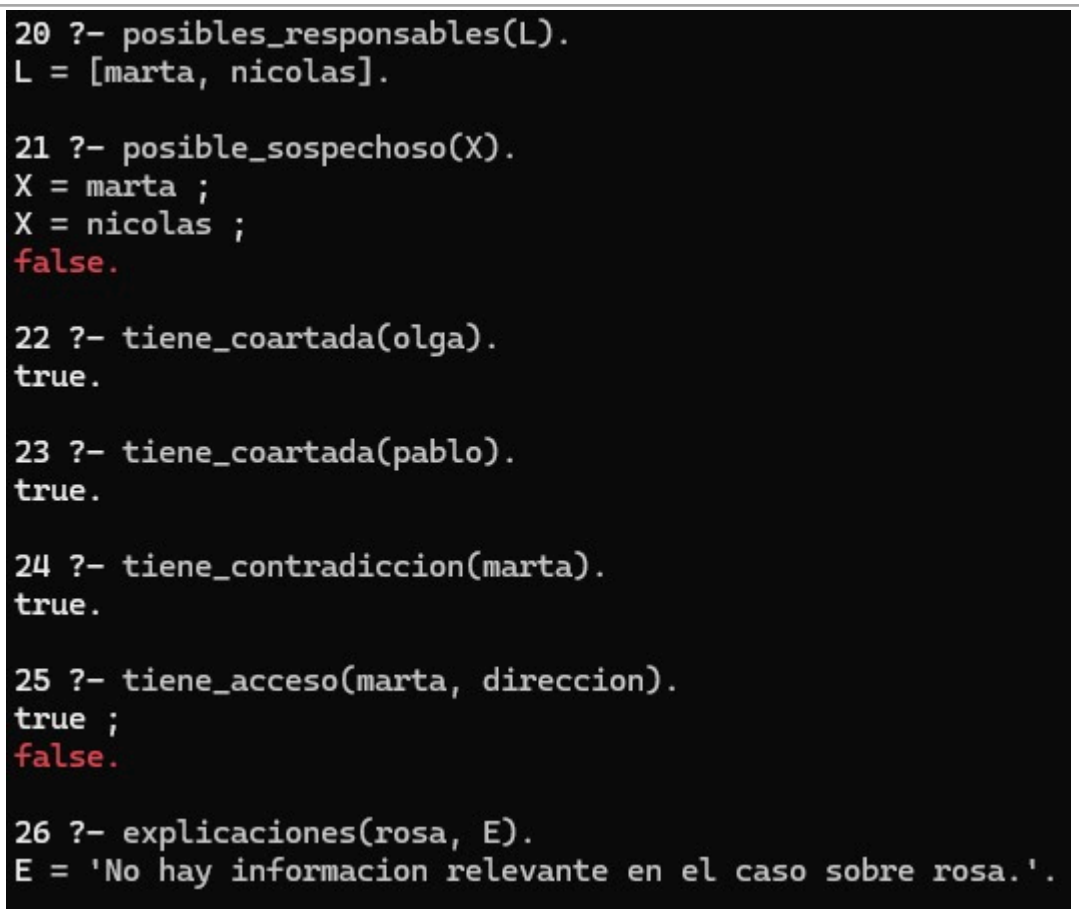


3.2 Lógica en Prolog – Caso 2 (escuela)

Segundo caso con otros datos (un robo en la dirección de una escuela). Se reemplazó hechos.pl sin modificar predicados.pl, para comprobar que las reglas no dependen de los datos. Este caso tiene dos sospechosos, dos personas con coartada y una persona sin movimientos registrados.

Captura C09 – Caso 2: dos sospechosos

Resultado esperado: L = [marta, nicolas], con , se obtienen ambos (backtracking).



Captura C10 – Caso 2: resolver_caso/0

Resultado esperado: Reporte completo de marta y de nicolas, ambos válidos.

```
27 ?- resolver_caso.
=== RESOLUCION DEL CASO ===
Evento: robo en direccion a las 1430

Posibles responsables: [marta,nicolas]

--- Responsable: marta ---
Evidencias relacionadas: []
Movimientos registrados:
  - a las 1400 en sala_profesores
  - a las 1420 en direccion
Lugares con acceso: [direccion,sala_profesores]
Acceso al lugar del evento (direccion): Si
Coartada: No
Contradiccion: Si (declaro sala_profesores a las 1425; un registro la ubica en direccion a las 1420)
Justificacion logica:
  - CONCLUSION: marta es posible sospechoso: estuvo en el lugar del evento durante la ventana, tuvo acceso o evidencia relacionada y no tiene coartada.
  - marta estuvo en direccion (lugar del evento) a las 1420, dentro de la ventana de 1330 a 1530.
  - marta tiene acceso a direccion, el lugar del evento.
  - marta declaro estar en sala_profesores a las 1425, pero un registro la ubica en direccion a las 1420: hay contradiccion.
Validacion de la solucion:
  - Estuvo en el lugar del evento durante la ventana: Si
  - Tuvo acceso o tiene evidencia relacionada: Si
  - No tiene coartada: Si
  => Solucion valida: Si

--- Responsable: nicolas ---
Evidencias relacionadas: [huella]
Movimientos registrados:
  - a las 1350 en patio
  - a las 1435 en direccion
Lugares con acceso: [direccion,patio]
Acceso al lugar del evento (direccion): Si
Coartada: No
Contradiccion: Si (declaro patio a las 1430; un registro la ubica en direccion a las 1435)
Justificacion logica:
  - CONCLUSION: nicolas es posible sospechoso: estuvo en el lugar del evento durante la ventana, tuvo acceso o evidencia relacionada y no tiene coartada.
  - nicolas estuvo en direccion (lugar del evento) a las 1435, dentro de la ventana de 1330 a 1530.
  - nicolas tiene acceso a direccion, el lugar del evento.
  - La evidencia huella esta relacionada con nicolas.
  - nicolas declaro estar en patio a las 1430, pero un registro la ubica en direccion a las 1435: hay contradiccion.
Validacion de la solucion:
  - Estuvo en el lugar del evento durante la ventana: Si
  - Tuvo acceso o tiene evidencia relacionada: Si
  - No tiene coartada: Si
  => Solucion valida: Si

true.
```

Captura C11 – Caso 2: persona sin movimientos

Resultado esperado: "No hay informacion relevante en el caso sobre rosa."

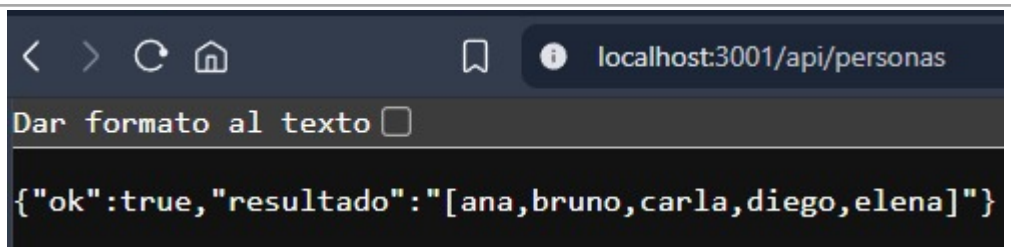
```
26 ?- explicaciones(rosa, E).
E = 'No hay informacion relevante en el caso sobre rosa.'.
```

3.3 Servidor (backend)

Se probaron las rutas del servidor desde el navegador. Todas responden con el mismo formato JSON: { "ok": true, "resultado": "..." } o { "ok": false, "error": "..." }.

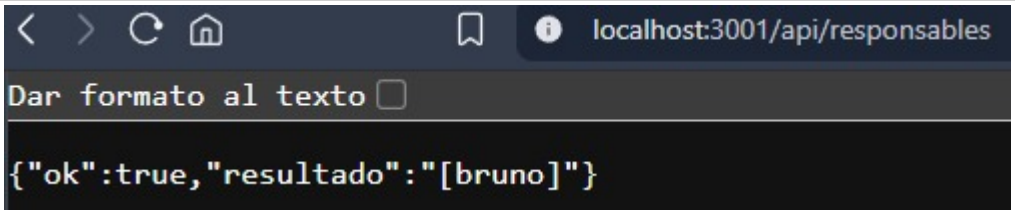
Captura C13 – Ruta /api/personas

Resultado esperado: [ana,bruno,carla,diego,elena]



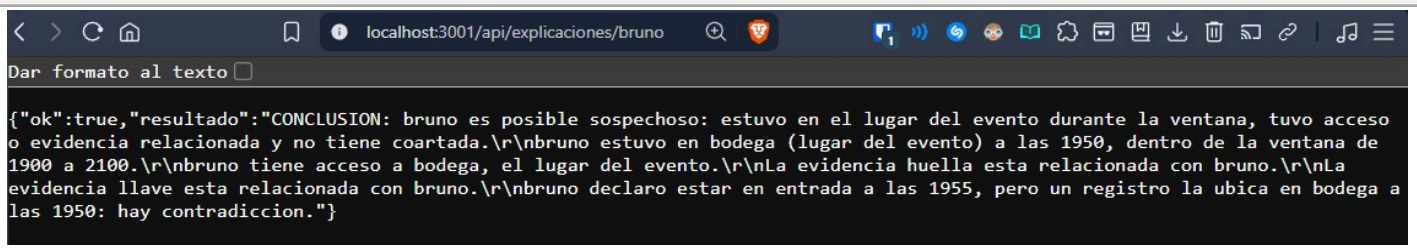
Captura C14 – Ruta /api/responsables

Resultado esperado: [bruno]



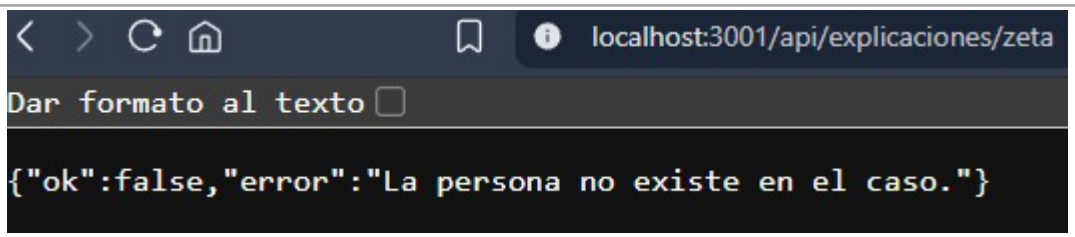
Captura C15 – Ruta /api/explicaciones/bruno

Resultado esperado: Las seis explicaciones, una por línea.



Captura C16 – Persona inexistente en /api/explicaciones/zeta

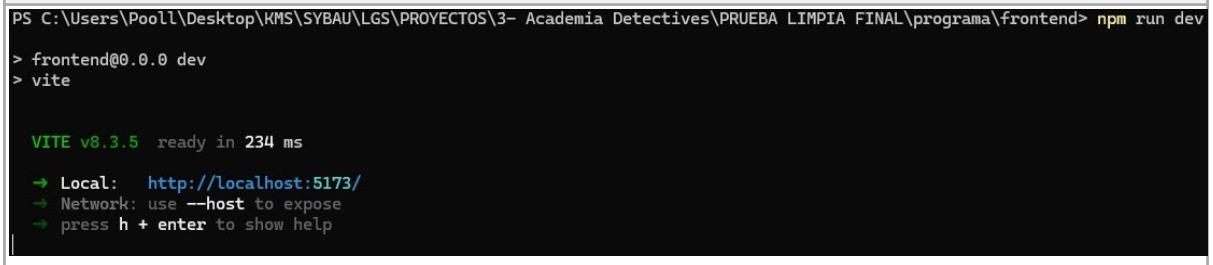
Resultado esperado: "La persona no existe en el caso." (error 404).



3.4 Interfaz (frontend)

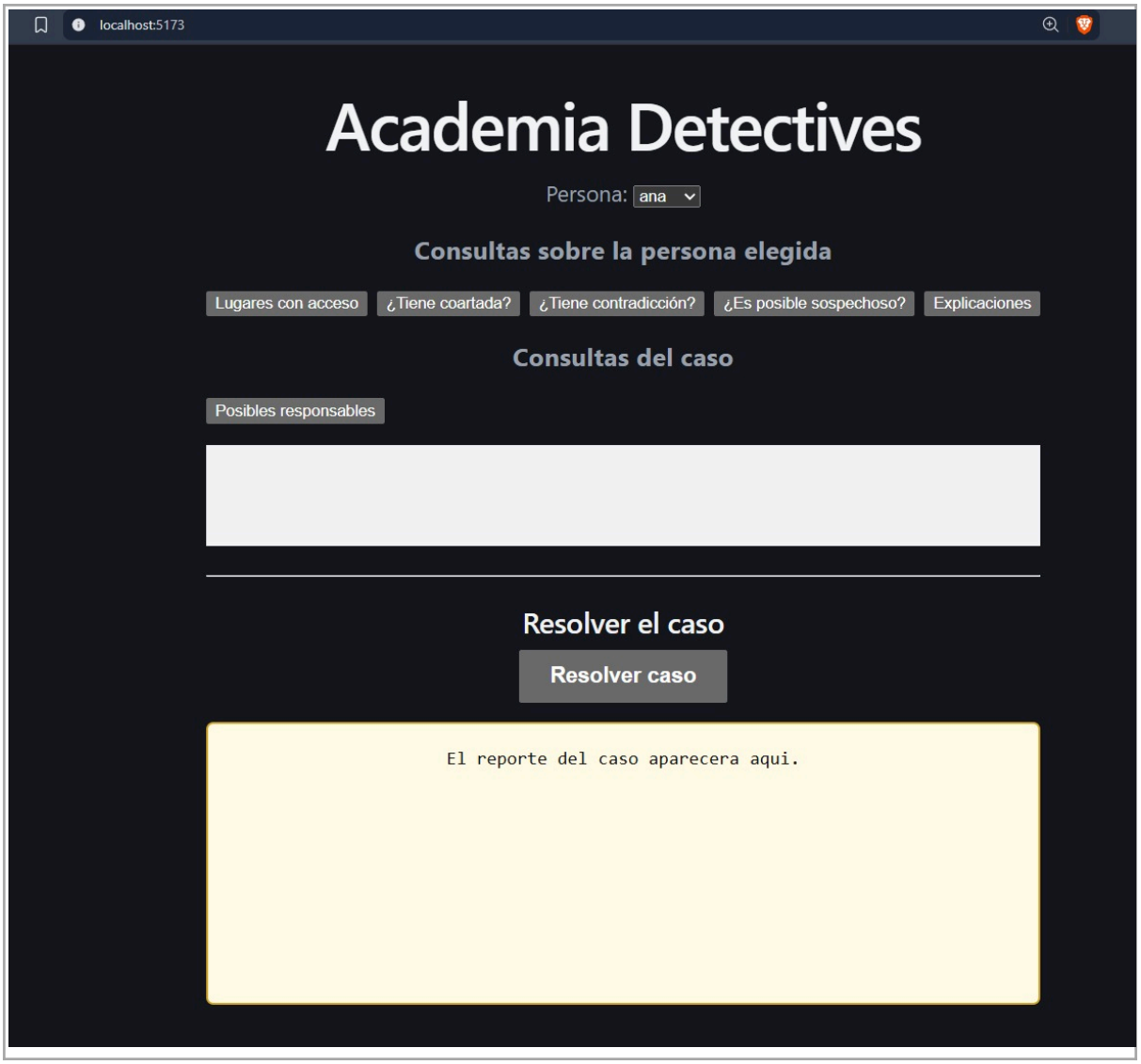
Captura C17 – Interfaz arrancada

Resultado esperado: Terminal de Vite con la dirección local.

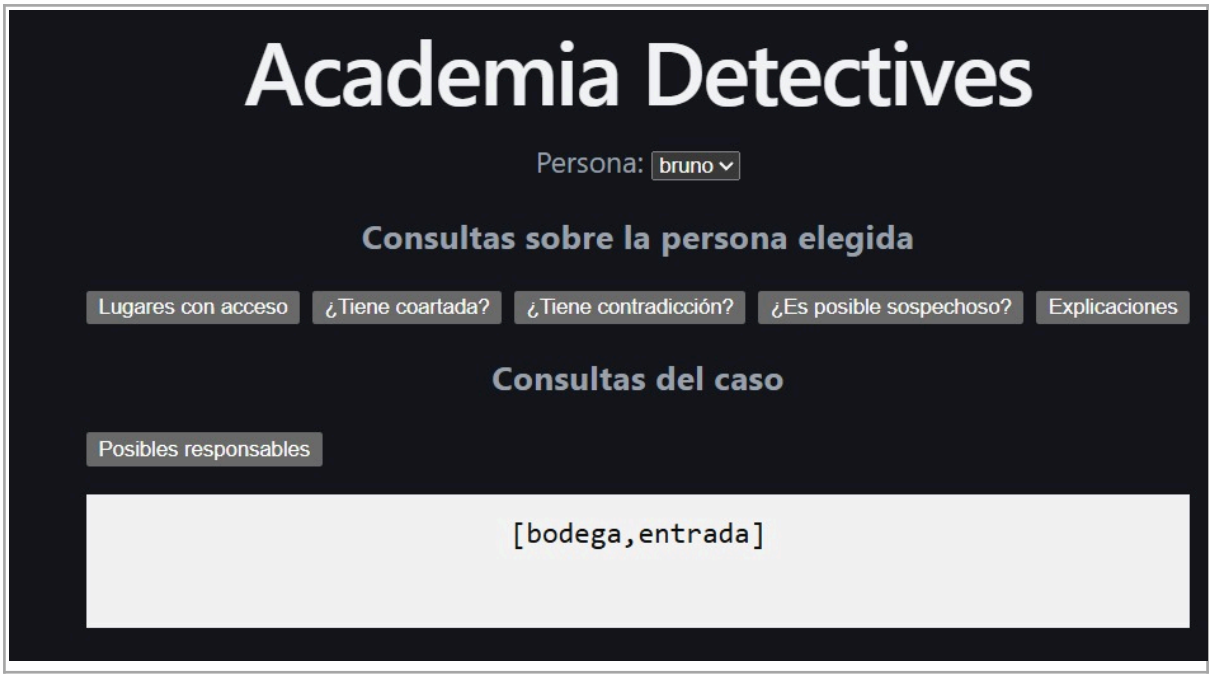


Captura C18 – Pantalla inicial

Resultado esperado: Lista de personas, botones de consulta y sección "Resolver el caso".

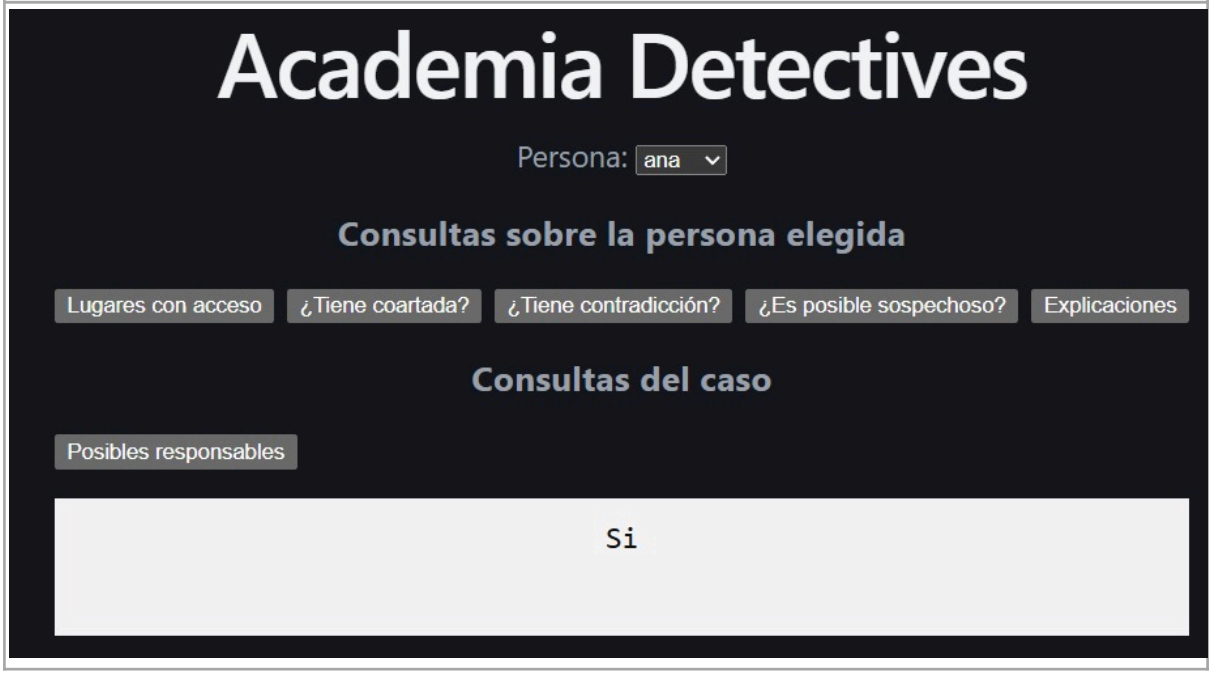


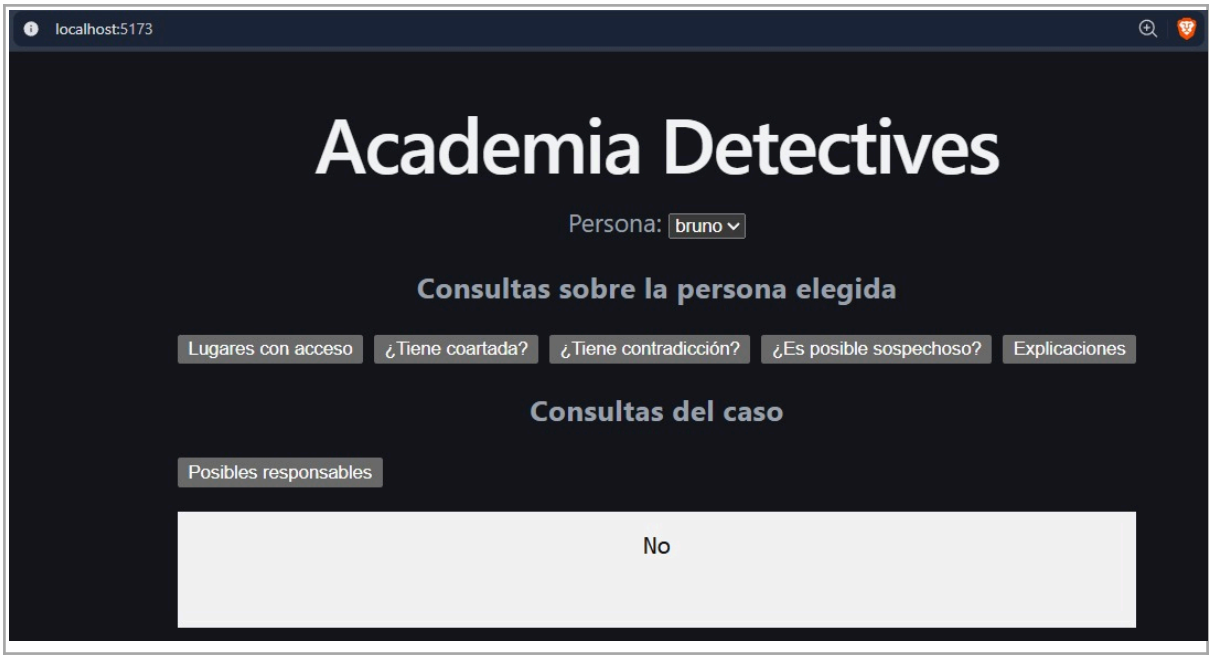
Captura C19 – Lugares con acceso (bruno)
Resultado esperado: [bodega,entrada]



Captura C20 – Coartada de ana y de bruno

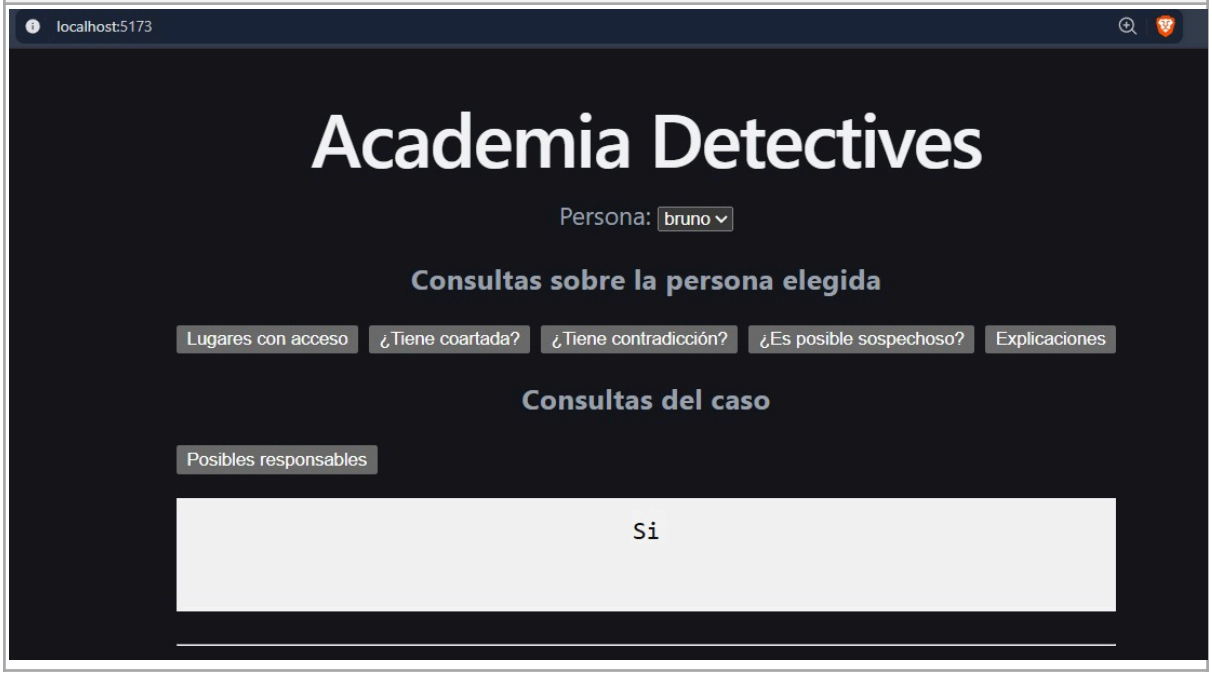
Resultado esperado: Si para ana, No para bruno.





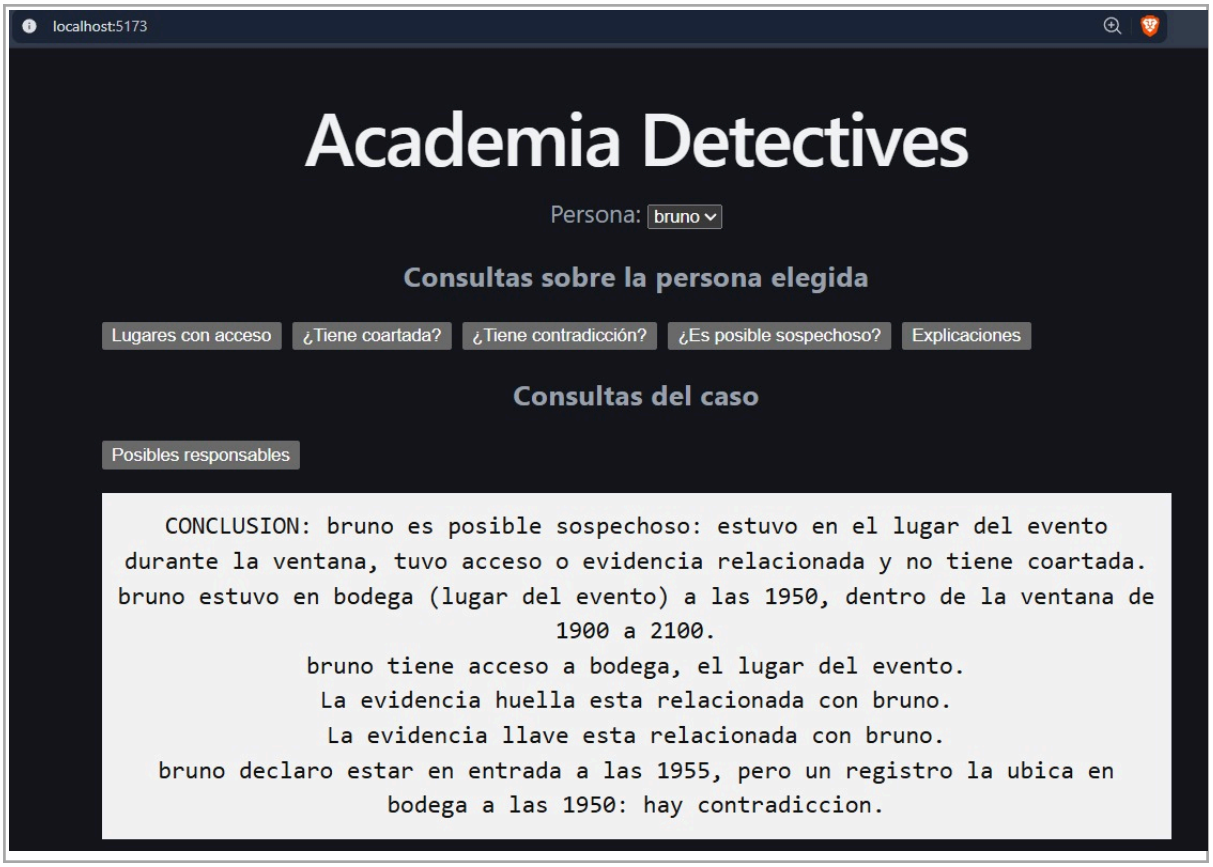
Captura C21 – ¿Es posible sospechoso? (bruno)

Resultado esperado: Si



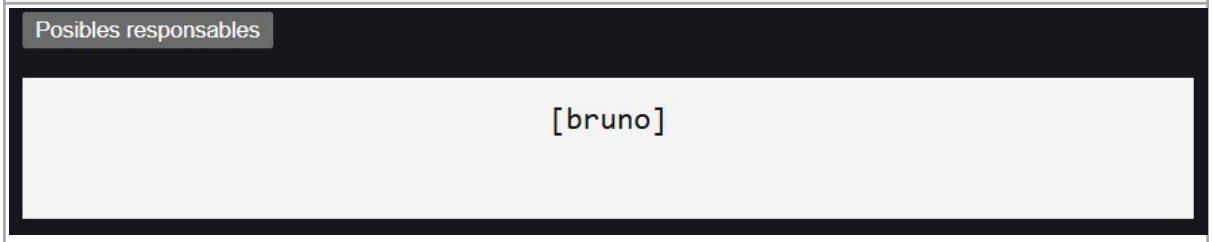
Captura C22 – Explicaciones (bruno)

Resultado esperado: Seis explicaciones, una por línea.



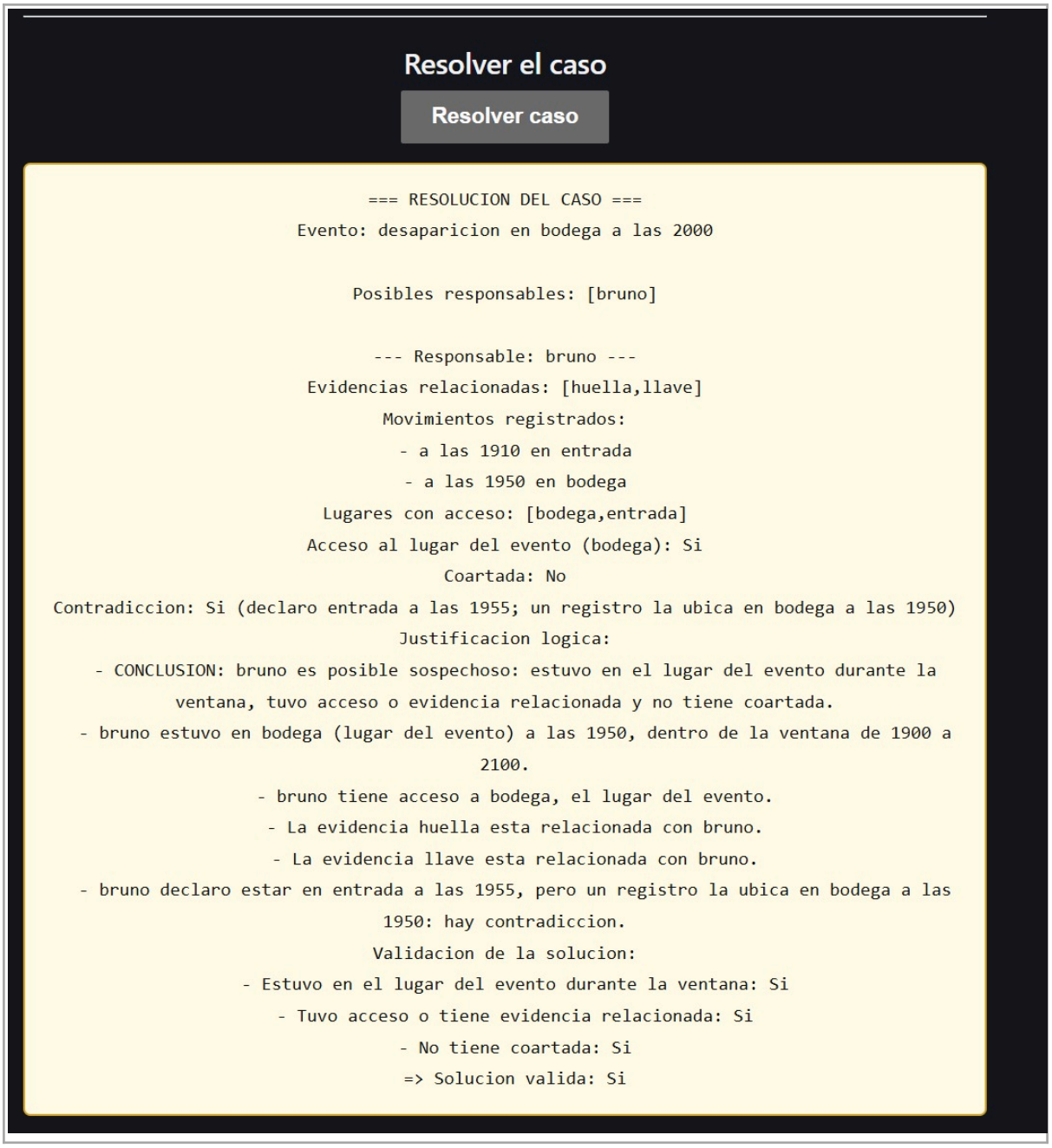
Captura C23 – Posibles responsables

Resultado esperado: [bruno]



Captura C24 – Resolver caso

Resultado esperado: Reporte completo de bruno en el recuadro amarillo.



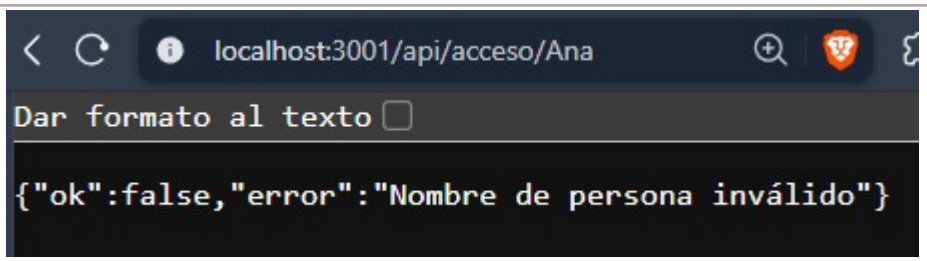
Captura C25 – Interfaz con el caso 2

Resultado esperado: Personas de la escuela, resolver caso con marta y nicolas.



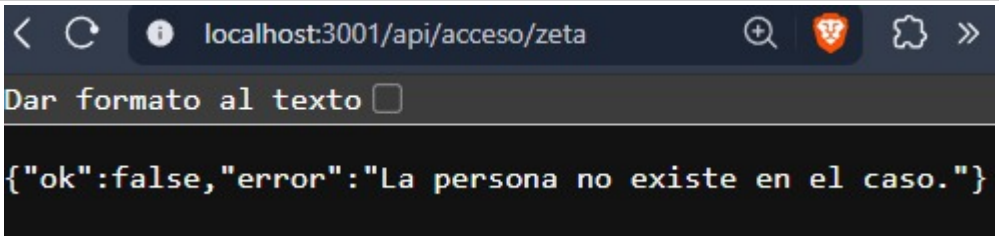
3.5 Manejo de errores

Captura C26 – Nombre inválido (/api/acceso/Ana)
Resultado esperado: "Nombre de persona inválido" (error 400).



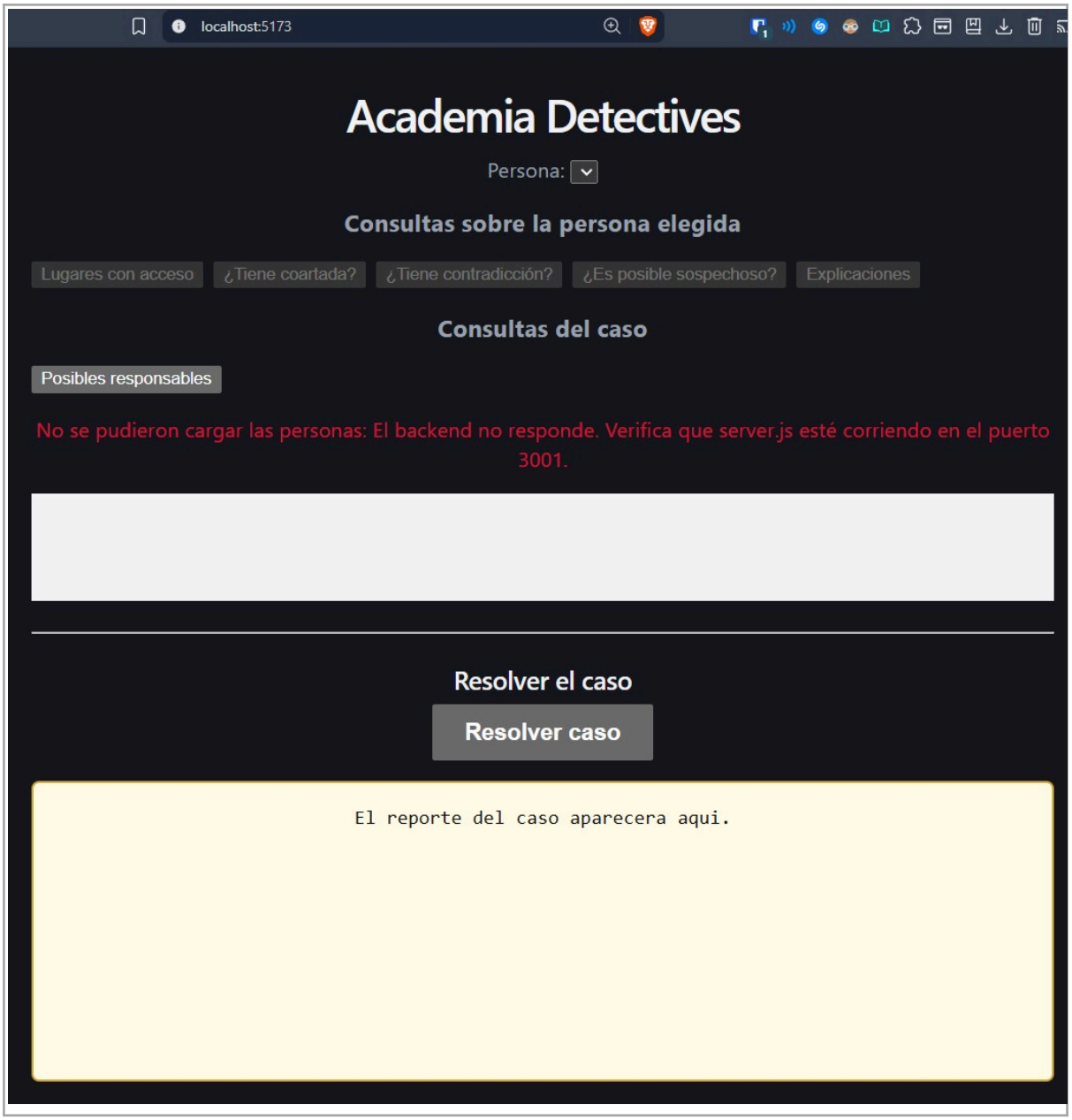
Captura C27 – Persona inexistente (/api/acceso/zeta)

Resultado esperado: "La persona no existe en el caso." (error 404).



Captura C28 – Interfaz con el servidor apagado

Resultado esperado: Mensaje en rojo indicando que el backend no responde.



4. Diseño del programa

4.1 Arquitectura

El sistema se divide en tres capas. Toda la lógica del caso está en Prolog, las otras dos capas solo transportan datos.

Capa	Tecnología	Responsabilidad

Interfaz	React con Vite (programa/frontend)	Recibe la persona elegida y la consulta solicitada, las envía al servidor y muestra el texto que devuelve.
Servidor intermedio	Node.js con Express, puerto 3001 (programa/backend/server.js)	Valida el nombre recibido, arma la meta de Prolog, ejecuta swipl y devuelve la salida en formato JSON. No decide nada del caso.
Lógica	SWI-Prolog (programa/backend/prolog)	hechos.pl guarda el caso, predicados.pl contiene las reglas que infieren las conclusiones.

Flujo de una consulta: el usuario pulsa un botón, React pide /api/... entonces el proxy de Vite lo reenvía al servidor luego el servidor ejecuta swipl -q -s predicados.pl -g Meta -t halt entonces Prolog imprime el resultado luego el servidor lo devuelve como JSON y con eso la interfaz lo muestra.

4.2 Separación entre hechos y reglas

El enunciado indica que durante la revisión se reemplazarán los hechos por los de otros casos. Por eso hechos.pl contiene solo hechos y predicados.pl no menciona ninguna persona, lugar ni hora concretos: todas las reglas se escriben con variables. Para que las reglas no fallen si un caso no trae algún tipo de hecho (por ejemplo, ubicado/2), predicados.pl declara los predicados de la base como dynamic, así Prolog responde false en lugar de lanzar un error de predicado inexistente.

4.3 Decisiones de diseño

1. **Ventana de tiempo del evento.** Se considera relevante el período de una hora antes y una hora después del evento (ventana_evento/3). Es la única definición de ventana: cambiar su tamaño implica modificar dos líneas.
2. **Comparación de horas.** Las horas están en formato HHMM, por lo que restarlas directamente da resultados incorrectos (2005 − 1955 da 50, pero son 10 minutos). El predicado auxiliar minutos/2 las convierte a minutos antes de compararlas.
3. **Acceso por objeto.** Una persona accede a un lugar protegido por un objeto si estuvo registrada en el lugar donde ese objeto se encuentra (ubicado/2), pues pudo obtenerlo.
4. **Coartada y contradicción inferidas.** Ninguna se guarda como hecho. La contradicción exige que un testimonio y un registro de la misma persona ubiquen a la persona en lugares distintos con una diferencia de 10 minutos o menos. La coartada exige un testimonio dentro de la ventana en un lugar distinto al del evento, sin contradicción.
5. **Un solo punto de verdad.** Los datos de una contradicción o coartada se obtienen con contradiccion_detalle/5 y coartada_detalle/3. Los usan tanto las reglas booleanas

como explicaciones/2 y resolver_caso/0, para que el texto mostrado siempre coincida con la regla aplicada.

6. **Sospechoso en tres condiciones.** Presencia, oportunidad y ausencia de coartada (ver sección 6).
7. **Salida de texto desde Prolog.** Los predicados que informan al usuario imprimen con format/2. El servidor solo captura esa salida, de modo que la interfaz no interpreta ni calcula nada.
8. **Seguridad del servidor.** Se usa execFile (no exec), de modo que el texto del usuario nunca pasa por una terminal. El nombre de la persona se valida con una expresión regular (minúscula inicial, luego letras, números o guion bajo), lo que impide inyectar variables o símbolos en la meta de Prolog. Si la persona no existe en persona/1, Prolog termina con el código 3 y el servidor responde con error 404. Hay un tiempo máximo de espera de 5 segundos.
9. **React con Vite.** Se eligió React por familiaridad del grupo con este mismo. El servidor de desarrollo de Vite reenvía las rutas /api al servidor, lo que evita configurar CORS.
10. **Predicados excluidos.** estuvo_en_intervalo/4, evidencias_de/2, descartar/1 e investigar/1 no se desarrollaron por indicación del profesor. Las reglas que los habrían usado consultan estuvo/3 y corresponde/2 directamente.

4.4 Técnicas y algoritmos usados

Técnica	Dónde y para qué
Unificación y backtracking	Todas las reglas. Con variables, Prolog prueba cada hecho y con, entrega todas las soluciones (por ejemplo, tiene_acceso(P, bodega) o posible_sospechoso(X)).
findall/3	Reunir listas: posibles responsables, accesos, evidencias y movimientos.
forall/2	Recorrer listas para imprimir (reporte de resolver_caso/0).
Negación por fallo (\+)	Expresar "no tiene contradicción" y "no tiene coartada".
once/1	Evitar que una persona salga repetida cuando una condición se cumple de varias formas.
Condicional (-> y ;)	Elegir el texto a mostrar (Si/No, con o sin coartada) sin que la regla falle.
sort/2	Quitar lugares repetidos y ordenar movimientos por hora.
Cláusulas múltiples	tiene_acceso/2 (dos vías) y explicacion/2 (una cláusula por tipo de explicación).

5. Análisis de resultados

5.1 Objetivos alcanzados

- Representación de un caso completo con hechos: personas, lugares, objetos, ubicaciones, accesos, accesos por objeto, movimientos, evento central, evidencias, relaciones evidencia-persona y testimonios.
- Reglas que infieren acceso, contradicción, coartada y posibles sospechosos sin guardar conclusiones como hechos.
- Explicaciones lógicas construidas con los datos reales del caso.
- resolver_caso/0 entrega, por cada responsable: evidencias, movimientos, accesos, coartada o contradicción, justificación lógica y validación de la solución.
- Uso de unificación y backtracking para obtener múltiples soluciones, comprobado con el caso 2 (dos sospechosos).
- Las mismas reglas funcionan con dos casos distintos sin modificar predicados.pl, y no fallan si falta algún tipo de hecho.
- Interfaz web en React enlazada a Prolog mediante un servidor Node.js, con validación de entradas y mensajes de error claros.
- Documentación interna (comentarios con entradas, salidas, restricciones y objetivo en cada predicado) y uso de GitHub durante el desarrollo.

5.2 Objetivos no alcanzados y razones

- **estuvo_en_intervalo/4, evidencias_de/2, descartar/1 e investigar/1:** no se desarrollaron porque el profesor indicó suprimirlos del proyecto (se conserva el mensaje como respaldo).
- **Documento de Requerimientos opcional:** No se llegó a entregar.

5.3 Limitaciones conocidas

- La ventana de tiempo es fija (una hora antes y después) y los eventos se suponen dentro de un mismo día, un evento cercano a la medianoche no se maneja.
- La contradicción se evalúa con una tolerancia de 10 minutos, una diferencia mayor no se considera contradicción.
- El acceso por objeto se infiere de la presencia de la persona donde está el objeto, no existe un hecho que indique quién posee cada objeto.
- El servidor inicia swipl en cada consulta. Es simple y seguro, aunque no es el método más rápido para un volumen alto de consultas.

6. Análisis de predicados

Esta sección explica las decisiones sobre los predicados pedidos en el enunciado. Se indican los ejemplos con el caso 1 (museo).

6.1 tiene_contradiccion/1

Propósito. Detectar cuando lo que declara una persona no coincide con los registros de movimientos, sin guardar un hecho que diga que "miente".

Decisión. Hay contradicción cuando existe un testimonio y un registro estuvo/3 de la misma persona que la ubican en lugares distintos con 10 minutos o menos de diferencia entre sus horas. Se exige que las horas estén cerca porque dos horas muy distintas no son incompatibles: la persona pudo moverse. Para medir la diferencia se convierten las horas a minutos con minutos/2.

```
contradiccion_detalle(Persona, LugarTestimonio, HoraTestimonio,
                      LugarRegistro, HoraRegistro) :-
        testimonio(Persona, LugarTestimonio, HoraTestimonio),
        estuvo(Persona, LugarRegistro, HoraRegistro),
        LugarTestimonio \= LugarRegistro,
        minutos(HoraTestimonio, MinutosTestimonio),
        minutos(HoraRegistro, MinutosRegistro),
        abs(MinutosTestimonio - MinutosRegistro) =< 10.

tiene_contradiccion(Persona) :-
        contradiccion_detalle(Persona, _, _, _, _).
```

Ejemplo. Bruno declaró estar en la entrada a las 1955, pero un registro lo ubica en la bodega a las 1950: hay contradicción. Carla declaró la biblioteca a las 1955 y un registro la ubica en el laboratorio a las 1955. Ana declaró la biblioteca a las 1930 y no hay registro que la contradiga.

6.2 posible_sospechoso/1

Propósito. Decidir si una persona puede considerarse posible sospechosa del evento central.

Decisión. Se piden tres condiciones a la vez, porque ninguna por sí sola alcanza:

14. **Presencia:** estuvo en el lugar del evento dentro de la ventana de tiempo (presente_en_evento/1). Sin presencia no hay oportunidad física.
15. **Oportunidad:** tenía acceso al lugar del evento o tiene alguna evidencia relacionada (tiene_oportunidad/1). Cubre quien pudo entrar y quien dejó rastro.
16. **Sin coartada:** no se puede inferir una coartada (\+ tiene_coartada/1).

Una contradicción refuerza la sospecha y aparece en la explicación, pero no se exige: alguien puede ser sospechoso sin haber declarado nada.

```
posible_sospechoso(Persona) :-
        persona(Persona),
        presente_en_evento(Persona),
        tiene_oportunidad(Persona),
        \+ tiene_coartada(Persona).
```

Se usa once/1 dentro de las condiciones auxiliares para que, al listar con una variable, cada persona salga una sola vez aunque cumpla una condición de varias formas.

Ejemplo. Bruno estuvo en la bodega a las 1950 (presencia), tiene acceso a la bodega y evidencias (oportunidad) y no tiene coartada: es sospechoso. Elena tiene acceso a la bodega (por la llave maestra) pero nunca estuvo allí: no es sospechosa.

6.3 descartar/1

Este predicado no se desarrolló por indicación del profesor. La conclusión contraria a "sospechoso" se refleja en el programa de otras formas: tiene_coartada/1 (coartada válida) y la validación que muestra resolver_caso/0, que indica qué condición de sospechoso no se cumple.

6.4 posibles_responsables/1

Propósito. Obtener la lista de personas que cumplen las condiciones de posible responsable.

Decisión. Se recorren todas las personas definidas con persona/1 y se conservan las que cumplen posible_sospechoso/1. Usar findall/3 garantiza que el predicado nunca falle: si no hay sospechosos, devuelve la lista vacía y resolver_caso/0 lo informa.

```
posibles_responsables(Lista) :-
        findall(Persona, (persona(Persona), posible_sospechoso(Persona)), Lista).
```

Ejemplo. Caso 1: [bruno]. Caso 2: [marta, nicolas].

6.5 explicaciones/2

Propósito. Justificar con hechos y reglas por qué una persona es sospechosa o relevante.

Decisión. Cada tipo de explicación es una cláusula independiente del auxiliar explicacion/2, y el texto se arma con format(atom(Texto), ...) usando los datos reales de la base. Nada se guarda como conclusión. Con backtracking se obtienen todas las explicaciones que aplican. Los tipos son:

- La conclusión de que la persona es posible sospechosa.
- Su presencia en el lugar del evento dentro de la ventana, con la hora.
- Su acceso al lugar del evento.
- Cada evidencia relacionada con ella.

- Su contradicción, con lo que declaró y lo que dice el registro.
- Su posible coartada, con el lugar y la hora declarados.

Si ninguna aplica, una segunda cláusula de explicaciones/2 (que usa \+ explicacion(Persona, _)) devuelve un texto indicando que no hay información relevante. Si la persona no existe en persona/1, el predicado falla, y el servidor lo informa como error 404.

Ejemplo. Para Bruno devuelve 6 explicaciones, para Ana, 2 (la evidencia del documento y su posible coartada), para Diego, el mensaje de que no hay información relevante.

6.6 resolver_caso/0

Propósito. Ejecutar el proceso completo y presentar la conclusión con su sustento.

Decisión. No se guarda ningún responsable: el predicado obtiene los responsables con posibles_responsables/1 y, para cada uno, imprime con reporte_responsable/1:

17. Evidencias relacionadas.
18. Ubicación y movimientos (ordenados por hora).
19. Accesos relevantes: lugares con acceso y si tenía acceso al lugar del evento.
20. Coartada y contradicción, con sus detalles.
21. Justificación lógica (las explicaciones de la persona).
22. Validación de la solución: se vuelven a comprobar por separado la presencia, la oportunidad y la ausencia de coartada, y se concluye si la solución es válida.

También maneja los casos especiales: si el caso no tiene evento o si no hay responsables, imprime un mensaje claro en lugar de fallar.

```
resolver_caso :-
        (   evento(Nombre, Lugar, Hora)
        ->  format('=== RESOLUCION DEL CASO ===~n'),
        format('Evento: ~w en ~w a las ~w~n~n', [Nombre, Lugar, Hora]),
        posibles_responsables(Responsables),
        (   Responsables == []
        ->  format('No se encontraron posibles responsables...~n')
        ;   format('Posibles responsables: ~w~n~n', [Responsables]),
        forall(member(Responsable, Responsables),
              reporte_responsable(Responsable))
        )
        ;   format('El caso no tiene un evento definido.~n')
        ).
```

Ejemplo. En el caso 1 el reporte concluye que Bruno es el responsable, con evidencias [huella, llave], acceso a la bodega, contradicción en su testimonio y "Solucion valida: Si".

7. Bitácora

La bitácora se genera a partir del historial de commits de GitHub. En el repositorio, en la carpeta de documentación se encuentra un documento llamado “bitacora.txt” en el que se hallaran todos los commits hechos en ese repositorio

Repositorio: Enlace al repositorio GitHub